

Performance Testing

Date	10 july 2026
Team ID	
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing & Performance Testing
Target Module / API	Emotion Prediction API (Flask Application)
Test Environment	Local Machine (Windows 11, Python 3.11, Flask)
Test Date	14 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user emotion prediction	1	60	Response generated successfully
2	Concurrent prediction requests	20	120	Stable response without failures
3	Continuous prediction requests	50	180	System maintains acceptable response time

4	Peak load prediction requests	100	300	No application crash; slight increase in response time
---	-------------------------------	-----	-----	--



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.24sec	pass	Meets target
2	Response Time (Max)	< 5 seconds	3.47sec	pass	Stable under peak load
3	Throughput (Req/sec)		28Req/sec	pass	Good performance
4	Error Rate	< 1%	0.2%	pass	Very few failed
5	CPU Utilization	< 80%	62%	pass	Efficient CPU usage
6	Memory Utilization	< 80%	68%	pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The Flask application successfully handled concurrent prediction requests.
- Average response time remained below the target limit.
- Emotion prediction models responded consistently under normal and peak workloads.
- No significant memory leaks or application crashes were observed during testing

Bottlenecks Identified

- Initial model loading increases the response time for the first request.
- Processing time increases slightly with higher concurrent users.
- AI recommendation generation via Gemini API depends on network latency

Optimization Steps Taken

- Loaded machine learning models only once during application startup.
- Optimized text preprocessing pipeline.
- Reduced redundant computations.
- Improved exception handling for invalid user inputs.
- Used efficient data structures to reduce memory usage.